

BRACKET · SECURITY REVIEW

# Bracket

# BracketFi Round 1.

A security review of BracketFi Round 1.

---

| CLIENT  | PROJECT           | AUDITORS | AUDIT WINDOW                      | FINAL REPORT     |
|---------|-------------------|----------|-----------------------------------|------------------|
| Bracket | BracketFi Round 1 | Guardian | December 19 –<br>January 13, 2025 | January 20, 2025 |

§ 00

# Contents.

49 pages — overview, confidence ranking, scope, 36 findings, disclaimer, and about Guardian.

|      |                                 |       |
|------|---------------------------------|-------|
| § 01 | Overview                        | P. 03 |
| § 02 | Confidence Ranking              | P. 04 |
| § 03 | Scope & Classifications         | P. 05 |
|      | Files in Scope                  | P. 06 |
| § 04 | Findings                        | P. 07 |
|      | Main Review · 28 findings       | P. 10 |
|      | Remediation Review · 8 findings | P. 39 |
| § 05 | Disclaimer                      | P. 48 |
| § 06 | About Guardian                  | P. 49 |

§ 01

Overview.

Guardian reviewed BracketFi Round 1 across 2 review rounds. The final report includes 3 Critical, 2 High, 9 Medium, 22 Low findings.

|              |                                |                          |                                                            |
|--------------|--------------------------------|--------------------------|------------------------------------------------------------|
| AUDIT FIRM   | Guardian                       | REPOSITORY               | bracket-fi/core-contracts                                  |
| CLIENT       | Bracket                        | METHODOLOGY              | Smart contract · Manual · Main Review · Remediation Review |
| PROJECT      | BracketFi Round 1              | MAIN REVIEW COMMIT       | 727a7751260e                                               |
| PREPARED BY  | Guardian                       | REMEDATION REVIEW COMMIT | 43af8f03f0d6                                               |
| AUDIT WINDOW | December 19 – January 13, 2025 |                          |                                                            |
| FINAL REPORT | January 20, 2025               |                          |                                                            |

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

|                     |                |                     |                 |                      |
|---------------------|----------------|---------------------|-----------------|----------------------|
| 1                   | 2              | 3                   | 4               | 5                    |
| VERY LOW CONFIDENCE | LOW CONFIDENCE | MODERATE CONFIDENCE | HIGH CONFIDENCE | VERY HIGH CONFIDENCE |

1.2 Findings at a Glance

| SEVERITY       | TOTAL | PENDING | ACK | PARTIAL | RESOLVED |
|----------------|-------|---------|-----|---------|----------|
| Critical       | 3     | 0       | 0   | 0       | 3        |
| High           | 2     | 0       | 1   | 0       | 1        |
| Medium         | 9     | 0       | 4   | 0       | 5        |
| Low            | 22    | 0       | 9   | 0       | 13       |
| Informational  | 0     | 0       | 0   | 0       | 0        |
| All severities | 36    | 0       | 14  | 0       | 22       |

## § 02

# Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

| CONFIDENCE RANKING                              | DEFINITION AND RECOMMENDATION                                                                                                                                                                                               | RISK PROFILE                                                                              |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>5 Very High Confidence</b>                   | Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit. | 0 High/Critical findings and few Low/Medium severity findings.                            |
| <b>4 High Confidence</b>                        | Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.    | 0-1 High/Critical findings per engagement week and little to no Medium severity findings. |
| <b>3 Moderate Confidence</b><br><b>ASSIGNED</b> | Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.                                                            | Some High/Critical findings, with various Medium severity findings.                       |
| <b>2 Low Confidence</b>                         | Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.                                                                     | Over 6 High and Critical findings across the engagement.                                  |
| <b>1 Very Low Confidence</b>                    | The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.                                                                           | Over 12 High and Critical findings across the engagement.                                 |

§ 03

# Scope & Classifications.

Guardian reviewed BracketFi Round 1 in `bracket-fi/core-contracts` . The scope includes 505 source lines across 4 files.

## 3.1 Severity Matrix

|            | IMP · HIGH | IMP · MED | IMP · LOW |
|------------|------------|-----------|-----------|
| LIK · HIGH | Critical   | High      | Medium    |
| LIK · MED  | High       | Medium    | Low       |
| LIK · LOW  | Medium     | Low       | Low       |

IMPACT · HIGH  
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM  
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW  
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH  
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM  
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW  
Unlikely to ever occur in production.

## Files in Scope.

| FILE                         | NSLOC | TOTAL |
|------------------------------|-------|-------|
| src/factory/VaultFactory.sol | 43    | 56    |
| src/token/BrktETH.sol        | 212   | 363   |
| src/vaults/RebasingToken.sol | 31    | 40    |
| src/vaults/BracketVault.sol  | 219   | 345   |

## § 04

## Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

| ID          | TITLE                                        | CATEGORY            | SEVERITY | STATUS       | PAGE  |
|-------------|----------------------------------------------|---------------------|----------|--------------|-------|
| MAIN REVIEW |                                              |                     |          |              |       |
| C-01        | Incorrect Amount Of Collateral Returned      | Logical Error       | CRITICAL | RESOLVED     | p. 10 |
| C-02        | Inaccurate Share Calculation On Mint         | Logical Error       | CRITICAL | RESOLVED     | p. 11 |
| C-03        | Incorrect Collateral Calculation During Burn | Logical Error       | CRITICAL | RESOLVED     | p. 12 |
| H-01        | Missing lastNavUpdate Update                 | Logical Error       | HIGH     | RESOLVED     | p. 13 |
| M-01        | Incomplete Check In calculateMint Function   | Validation          | MEDIUM   | RESOLVED     | p. 14 |
| M-02        | Decreased NAV Frontrunning                   | Frontrunning        | MEDIUM   | RESOLVED     | p. 16 |
| M-03        | Manager Avoids Negative Performance Fees     | Unexpected behavior | MEDIUM   | ACKNOWLEDGED | p. 17 |
| M-04        | Wrong Index For Whitelist/Blacklist          | Logical Error       | MEDIUM   | RESOLVED     | p. 18 |
| M-05        | Incorrect Withdrawable Assets                | Logical Error       | MEDIUM   | RESOLVED     | p. 19 |
| M-06        | totalValue DoS                               | DoS                 | MEDIUM   | ACKNOWLEDGED | p. 20 |
| M-07        | Manager Fees Withdrawn By Users              | Logical Error       | MEDIUM   | RESOLVED     | p. 21 |
| L-01        | Swap Can Have Equal In and Out Tokens        | Validation          | LOW      | RESOLVED     | p. 22 |
| L-02        | Misvaluation Due to stETH-ETH Peg Assumption | Oracles             | LOW      | ACKNOWLEDGED | p. 23 |
| L-03        | Fees Unavailable For Claim                   | Logical Error       | LOW      | ACKNOWLEDGED | p. 24 |

## Findings continued.

| ID   | TITLE                                       | CATEGORY            | SEVERITY | STATUS       | PAGE  |
|------|---------------------------------------------|---------------------|----------|--------------|-------|
| L-04 | Lack of Slippage Control                    | Slipagge            | LOW      | RESOLVED     | p. 25 |
| L-05 | Unused Params                               | Optimization        | LOW      | RESOLVED     | p. 26 |
| L-06 | Increased Gas Cost For Multiple Collaterals | Configuration       | LOW      | ACKNOWLEDGED | p. 27 |
| L-07 | Adding Collateral Without Oracle Support    | Validation          | LOW      | RESOLVED     | p. 28 |
| L-08 | Vault Does Not Validate Zero Amounts        | Validation          | LOW      | RESOLVED     | p. 29 |
| L-09 | Missing Admin Functions                     | Configuration       | LOW      | RESOLVED     | p. 30 |
| L-10 | Unlicensed Smart Contracts                  | Best practices      | LOW      | RESOLVED     | p. 31 |
| L-11 | balanceOf Includes Pending Amounts          | Unexpected behavior | LOW      | ACKNOWLEDGED | p. 32 |
| L-12 | Vanity Nav Updated In Epoch 0               | Validation          | LOW      | RESOLVED     | p. 33 |
| L-13 | Lacking Pause Mechanism                     | Best practices      | LOW      | RESOLVED     | p. 34 |
| L-14 | Unnecessary Address(0) Check                | Optimization        | LOW      | RESOLVED     | p. 35 |
| L-15 | Excessive Wait For Manager Withdrawals      | Logical Error       | LOW      | RESOLVED     | p. 36 |
| L-16 | Collateral DoS Risk                         | DoS                 | LOW      | RESOLVED     | p. 37 |
| L-17 | Lacking Slippage Check Allows Griefing      | Griefing            | LOW      | RESOLVED     | p. 38 |

### REMEDIATION REVIEW

|        |                                                |                |        |              |       |
|--------|------------------------------------------------|----------------|--------|--------------|-------|
| H-01   | Missing collateralsIndex Update                | Logical Error  | HIGH   | ACKNOWLEDGED | p. 39 |
| M-01   | Invalid Index Checks                           | Logical Error  | MEDIUM | ACKNOWLEDGED | p. 40 |
| M-02   | lastNavUpdate Updated Before Conditional Check | Logical Error  | MEDIUM | ACKNOWLEDGED | p. 41 |
| L-01   | Missing Libraries Initialization               | Best Practices | LOW    | ACKNOWLEDGED | p. 42 |
| L-01.2 | Typo                                           | Typo           | LOW    | ACKNOWLEDGED | p. 43 |

## Findings continued.

| ID     | TITLE                                                                      | CATEGORY | SEVERITY | STATUS       | PAGE  |
|--------|----------------------------------------------------------------------------|----------|----------|--------------|-------|
| L-02   | Unimplemented _getPythPrice Function                                       | Warning  | LOW      | ACKNOWLEDGED | p. 44 |
| L-02.2 | Custom Revert Will Not Be Reached                                          | Warning  | LOW      | ACKNOWLEDGED | p. 45 |
| L-03   | _getChainlinkPrice Function Is Only Compatible With 8 decimals Price Feeds | Warning  | LOW      | ACKNOWLEDGED | p. 46 |

CRITICAL **C-01**

RESOLVED

# Incorrect Amount Of Collateral Returned

CATEGORY Logical Error  
LOCATION BrktETH.sol  
REVIEW Main Review

## DESCRIPTION

The existing `calculateBurn` logic incorrectly calculates how much of the `token` should be returned to the user for the given amount of brktETH value.

For example:

- (1) User deposits 10 WSTETH and receives 10 brktETH. Assume this is the entire brktETH supply.
- (2) ETH rate of WSTETH goes from 1 ETH to 1.1 ETH.
- (3) User burns all 10 brktETH shares, but does not receive 10 WSTETH. Instead they get  $(1.1 \text{ ether} * 10 \text{ ether} / 1 \text{ ether})$  which is 11 ether, more than the 10 WSTETH they put in.

This can be extremely detrimental to protocol as a user may extract more funds than appropriate, as well as prevent a depositor from withdrawing all their shares. Furthermore, this issue will occur every single time `burn` is called. To accurately calculate how much `token` should be returned, instead the ETH value of the redeemed brktETH should be divided by the ETH value of 1 `token`.

## RECOMMENDATION

Change the calculation to `return Math.mulDiv(value, 1 ether, oracle.getRate(token));`

PROOF OF CONCEPT <https://github.com/GuardianAudits/bracket-1/pull/1>

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/4efa7d75865b37e68150d357b10ce32aef1cfa6e>

## CRITICAL C-02

RESOLVED

# Inaccurate Share Calculation On Mint

|          |                 |
|----------|-----------------|
| CATEGORY | Logical Error   |
| LOCATION | BrktETH.sol: 64 |
| REVIEW   | Main Review     |

## DESCRIPTION

The `mint` function in the `BrktETH` contract currently deposits tokens into the contract—updating the token's `totalDeposit`—before calculating the amount to mint. As a result, the user's newly deposited tokens are included in the total value during the share calculation, causing the user to receive fewer shares than intended.

For example:

- (1) Alice deposits 10 WSTETH and mints 10 brktETH
- (2) Bob deposits 10 WSTETH directly after, and mints 5 brktETH ( $10 \text{ brktETH} * 10 \text{ ETH} / 20 \text{ ETH}$ ), although he should receive the same amount of brktETH as Alice due to 50-50% supply of the pool.

Consequently, this causes loss of assets for the depositor as they receive less shares than necessary, and will occur each time after the first mint.

## RECOMMENDATION

Modify the mint function so that it calculates the amount of tokens to be minted before depositing them into the contract.

**PROOF OF CONCEPT** <https://github.com/GuardianAudits/bracket-1/blob/bc53b44611ee51f90ac0565d2abdeb4b3b2b75db/test/guardian/GuardianPoCs.sol#L48>

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/4a2dc962897855610b365e9869d23fde864445a5>

CRITICAL **C-03**

RESOLVED

# Incorrect Collateral Calculation During Burn

CATEGORY Logical Error  
LOCATION BrktETH.sol: 79  
REVIEW Main Review

## DESCRIPTION

The burn function currently burns the user's brktETH tokens before calculating the amount of collateral they are entitled to. By doing so, the calculation for `colAmount` takes place after the user's share has already been removed from the supply, which may cause them to receive more collateral than they should or end up with nothing at all if they are the only remaining shareholder.

## RECOMMENDATION

Modify the burn function to calculate the collateral amount before burning the user's brktETH.

PROOF OF CONCEPT <https://github.com/GuardianAudits/bracket-1/blob/Incorrect-Collateral-Calculation-During-Burn/test/guardian/IncorrectBurnCalcSharesPOC.t.sol>

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/4a2dc962897855610b365e9869d23fde864445a5>

HIGH **H-01**

RESOLVED

# Missing lastNavUpdate Update

CATEGORY Logical Error  
LOCATION BracketVault.sol: 117  
REVIEW Main Review

## DESCRIPTION

In the `updateNav` function there is no update to the `lastNavUpdate` variable, therefore the confirmation period validation will never apply past the initial 1 day period after the vault starts.

## RECOMMENDATION

Update the `lastNavUpdate` variable in the `updateNav` function

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/cce9ce59f262f0e405b768012120d86d0427cf56>

MEDIUM **M-01**

RESOLVED

# Incomplete Check In calculateMint Function

CATEGORY Validation  
LOCATION BrktETH.sol: 242  
REVIEW Main Review

## DESCRIPTION

The current `calculateMint` implementation assumes that if `totalSupply` is zero, there is no existing pool of assets and thus sets `brktAmount = value`:

```
function calculateMint(uint256 value) public view returns (uint256 brktAmount) {  
    uint256 supply = totalSupply();  
    if (supply != 0) {  
        brktAmount = Math.mulDiv(supply, value, getTotalValue());  
    } else {  
        brktAmount = value;  
    }  
}
```

However, in a very specific edge case scenario, it could be possible that `totalSupply` is non-zero but `getTotalValue` is actually zero, for example, due to a sudden asset devaluation. In that case, a division by zero would occur blocking any new deposits (once the issue where you deposit before calculating the minted `brktAmount` is corrected).

## RECOMMENDATION

Update the `calculateMint` function as shown below:

```
function calculateMint(uint256 value) public view returns (uint256 brktAmount) {  
    uint256 supply = totalSupply();  
    uint256 totalValue = getTotalValue();  
    if ((supply != 0) && (totalValue != 0)) {  
        brktAmount = Math.mulDiv(supply, value, totalValue);  
    } else {  
        brktAmount = value;  
    }  
}
```

**M-01** Incomplete Check In calculateMint Function

CONTINUED

**— RESOLUTION**

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/a7ed83faddbcc817b51715d66867fd830cdc7eeb>

MEDIUM **M-02**

RESOLVED

# Decreased NAV Frontrunning

CATEGORY Frontrunning  
LOCATION BracketVault.sol  
REVIEW Main Review

## DESCRIPTION

If the NAV\_UPDATER calls `updateNav` with a `newNav` that is lower than the previous one, a user can frontrun the update and trigger a withdrawal. If the delay is only 1 epoch, then the user's shares will be valued at the previous epoch's NAV which is higher, withdrawing more assets and avoiding the loss.

## RECOMMENDATION

Consider enforcing that the delay is greater than 1 epoch, otherwise clearly document this risk and using a private RPC.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/90315e607b28302f5da35df3d5bb3609a35576bd>

MEDIUM **M-03**

ACKNOWLEDGED

# Manager Avoids Negative Performance Fees

CATEGORY Unexpected behavior  
LOCATION BracketVault.sol: 175  
REVIEW Main Review

## DESCRIPTION

The `claimManagerPerformanceFees` function may be called once every 90 days and each time the `accruedManagerPerformanceFees` are reset to zero.

This way a manager does not have to continue to pay down a large negative performance fee if it has accrued and has been being paid off for a long duration of time.

However if a manager submits an `updateNav` call with a negative performance fee directly before the 90 period is over they can almost immediately avoid the negative performance fee by calling the `claimManagerPerformanceFees` function again, whether on purpose or by accident.

## RECOMMENDATION

Consider using a separate interval for clearing negative performance fees which resets every time a new negative performance fee is added.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

# Wrong Index For Whitelist/Blacklist

CATEGORY Logical Error  
LOCATION BrktEth.sol  
REVIEW Main Review

## DESCRIPTION

Within functions `whitelistCollateral` and `blacklistCollateral` the index of the token is retrieved to whitelist/blacklist the token respectively. The issue is that the `collateralsIndex` starts from 1, so the whitelist/blacklist will be set for the wrong token: `uint256 index = collateralsIndex[token];`

## RECOMMENDATION

Use function `_getIndex` instead since it subtracts the returned index by 1.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/6853354537c04b50d4f82193e489a0ce1d377cac>

MEDIUM **M-05**

RESOLVED

# Incorrect Withdrawable Assets

CATEGORY Logical Error  
LOCATION BracketVault.sol: 293  
REVIEW Main Review

## DESCRIPTION

The `BracketVault.withdrawableAssets()` should allow users to query the max amount of assets they can withdraw from the vault.

However, this function will incorrectly subtract pending deposit shares when the `lastDeposit` has occurred in a previous epoch. The assets for a `lastDeposit` that occurred in a previous epoch should be represented as withdrawable though since they will be minted to the user in the `withdraw` function.

## RECOMMENDATION

Modify the `withdrawableAssets` view function such that the `pendingDepositShares` are only removed from the withdrawable amount when the epoch of the `lastDeposit` is the same as the current epoch:

```
function withdrawableAssets(address account) external view returns (uint256) {  
    uint256 lastNav = epoch == 0 ? 1e18 : navs[epoch - 1];  
    Deposit memory _deposit = lastDeposit[user];  
    if (_deposit.epoch == epoch) return convertToAssets(sharesOf(account) -  
        convertToShares(_deposit.assets, vanityNav), lastNav);  
    else return convertToAssets(sharesOf(account), lastNav);  
}
```

PROOF OF CONCEPT <https://github.com/GuardianAudits/bracket-2/blob/guardian/test/guardian/tests/GuardianTests.t.sol#L219>

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/bf3f70b9580b631655df95be60a21219c4a916d9>

MEDIUM **M-06**

ACKNOWLEDGED

## totalValue DoS

|          |                       |
|----------|-----------------------|
| CATEGORY | DoS                   |
| LOCATION | Bracket0racle.sol: 46 |
| REVIEW   | Main Review           |

### — DESCRIPTION

The BrktETH contract relies on the `getTotalValue` function to determine the total value of all collateral denominated in ETH, which is critical for functions like minting and burning. The process involves fetching each collateral's value and summing the results. However, if any one token's price retrieval fails, the entire `getTotalValue` function reverts—causing all dependent operations (e.g., minting, burning) to fail as well. For instance, the EZETH token uses an external oracle that performs certain validations when fetching its price, which could revert if those validations fail. This creates a single point of failure that disrupts the contract's functionality whenever a single token's price feed encounters an issue.

### — RECOMMENDATION

Implement a fallback oracle for each token so that if the primary method reverts (e.g., due to external validation errors), the contract can still fetch the token's price. This prevents the entire `getTotalValue` function—and consequently critical operations like minting or burning—from halting when a single price feed fails.

### — RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

RESOLVED

# Manager Fees Withdrawn By Users

CATEGORY Logical Error  
LOCATION BracketVault.sol: 260  
REVIEW Main Review

## DESCRIPTION

In the `_processDepositsWithdrawals` function the available balance for withdrawal by the users is based on the `_getManagerAvailableBalance`, which does not set aside the `accruedManagerPerformanceFees` (if positive), `accruedManagerTvlFees`, and `accruedBrktTvlFees`.

If a manager has not claimed these fees in a significant amount of time, but then submits an `updateNav` where they cannot fully cover all withdrawals for this epoch then the full manager balance or approval amount is used up.

The fee amounts will still be tracked in the `accruedBrktTvlFees`, `accruedManagerTvlFees`, and `accruedManagerPerformanceFees` variables, but the manager will not be able to claim these amounts as the `brktEth` and `brktEth` approval has been removed from the manager address.

## RECOMMENDATION

Consider reducing the available amount by the `accruedManagerPerformanceFees` (if positive), `accruedManagerTvlFees`, and `accruedBrktTvlFees` in the `_processDepositsWithdrawals` function.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/ffc20bb03f8e66f396e3e4f9b43bee41855b1dbc>

LOW **L-01**

RESOLVED

# Swap Can Have Equal In and Out Tokens

CATEGORY Validation  
LOCATION BrktETH.sol: 155  
REVIEW Main Review

## DESCRIPTION

Function `swapCollateral` lacks validation that `tokenIn != tokenOut`. In such a case, there could be an early return not to waste gas on calculating the rebalance and transfers.

## RECOMMENDATION

Early return if `tokenIn == tokenOut`

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/c78455d6b3bc6a36d9f9f856b160c119eda7233a>

LOW **L-02**

ACKNOWLEDGED

# Misvaluation Due to stETH-ETH Peg Assumption

CATEGORY Oracles  
LOCATION Bracket0racle.sol: 58  
REVIEW Main Review

## DESCRIPTION

The `_getWstethRate` function calls the `stEthPerToken()` function in the WSTETH contract, which returns the amount of stETH per wstETH. This value is then treated as if stETH were pegged 1:1 to ETH. However, stETH can and has previously depegged from ETH ([Ref](https://dune.com/LidoAnalytical/Curve-ETHstETH)), making this assumption unreliable.

## RECOMMENDATION

Use a reliable price feed, such as Chainlink's stETH-ETH feed ([Link](https://data.chain.link/feeds/ethereum/mainnet/steth-eth)), to accurately determine the ETH value instead of assuming a one-to-one peg.

## RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

# Fees Unavailable For Claim

|          |                  |
|----------|------------------|
| CATEGORY | Logical Error    |
| LOCATION | BracketVault.sol |
| REVIEW   | Main Review      |

## DESCRIPTION

Over time fees are accrued within the BracketVault: `accruedManagerPerformanceFees`, `accruedManagerTvlFees`, and `accruedBrktTvlFees`.

However, there is no guarantee that there is sufficient brktETH balance for the fees to be claimed because user withdrawals may decrease the manager's balance.

For example:

1. Nav is 1 ether,
2. Alice deposits 10 brktETH and gets 10 shares. She is the sole depositor.
3. Nav increases to 2 ether, with >0 fees accumulated.
4. Alice burns 10 shares to get 20 brktETH, which comes from the manager's balances.
5. Fee claim is attempted but manager does not have sufficient balance for the fee and causes a `ERC20InsufficientBalance` revert.

## RECOMMENDATION

Ensure managers are aware to set aside brktETH for fees.

## RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

# Lack of Slippage Control

|          |                 |
|----------|-----------------|
| CATEGORY | Slipagge        |
| LOCATION | BrktETH.sol: 64 |
| REVIEW   | Main Review     |

## DESCRIPTION

The mint and burn functions in the BrktETH contract rely on the vault's total collateral value to determine the number of shares or collateral token a user will receive. Because the vault uses tokens that can change in value at any time—for example, through rebases or slashings. This can lead to unexpected outcomes, such as the user ending up with fewer shares minted, or returning a different ratio of collateral when burning tokens.

## RECOMMENDATION

Implement slippage control that allows users to specify acceptable thresholds.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/b2c2c4a176d4633dda724ad92eb3dddfbf10e36>

LOW **L-05**

RESOLVED

## Unused Params

|          |                  |
|----------|------------------|
| CATEGORY | Optimization     |
| LOCATION | BracketVault.sol |
| REVIEW   | Main Review      |

### DESCRIPTION

The following param/error is not used in `BracketVault` contract:

- `feeClaimer` state variable
- `DepositIsCurrentEpoch` error

### RECOMMENDATION

Remove unused code or consider adding it to the current implementation.

### RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/8637d47560a08313a1a552a3fd8fd07f163adb31>

LOW **L-06**

ACKNOWLEDGED

# Increased Gas Cost For Multiple Collaterals

|          |                  |
|----------|------------------|
| CATEGORY | Configuration    |
| LOCATION | BrktETH.sol: 224 |
| REVIEW   | Main Review      |

## — DESCRIPTION

The `BracketOracle` currently supports 10 collateral tokens. In case all 10 are added and deposits are non zero, the `getTotalValue` function will need to iterate through every token to fetch the rate, adding around 270,000 gas, with Renzo's Staked ETH rate being the most gas intensive.

As a result, main actions will have an increased gas consumption:

- burn: 325671
- mint: 362535

As the protocol is deployed on the Ethereum mainnet , this gas cost can become a barrier for users to invest in the `BrktEth` vault.

## — RECOMMENDATION

Consider this scenario when adding supported collaterals.

## — RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

# Adding Collateral Without Oracle Support

CATEGORY Validation  
LOCATION BrktETH.sol: 203  
REVIEW Main Review

## DESCRIPTION

There are ten collaterals currently supported in `BracketOracle`, but `BrktETH.addCollateral` does not check if the collateral being added is currently supported.

As the `addCollateral` is only called once, it will be wise to make a call to `getRate(token)` to ensure its supported.

In the future, when new collaterals are added (i.e. apxETH, ETHx), this check will ensure `BracketOracle` is updated first.

## RECOMMENDATION

When adding a new collateral, consider executing `getRate(token)` to verify if the oracle supports it.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/c7c0423358773a5a919f6094f7ccd76942121685>

LOW **L-08**

RESOLVED

# Vault Does Not Validate Zero Amounts

CATEGORY Validation  
LOCATION BracketVault.sol: 198  
REVIEW Main Review

## DESCRIPTION

During `deposit` and `withdraw`, the `assets` param is not validated, so it can be 0. Although there is no major impact, the function does not revert and events are emitted, potentially causing issues in the UI.

## RECOMMENDATION

Consider validating for zero asset amount during `deposit` and `withdraw`.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/50e4814932bb76ba1dbbac9210d37c4f2dee895b>

LOW **L-09**

RESOLVED

# Missing Admin Functions

CATEGORY Configuration  
LOCATION BracketVault.sol: 90  
REVIEW Main Review

## DESCRIPTION

The `manager` address is set during vault initialization. However, if there is an issue with manager or it's compromised, there is no admin function to update this address.

Similarly, `withdrawal` delay is set at initialization but can't be updated again.

## RECOMMENDATION

Consider adding admin functions to update `manager` and `withdrawalDelay`.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/c152859e08356daf76aab3776597b61f7c1589d5>

LOW **L-10**

RESOLVED

# Unlicensed Smart Contracts

|          |                |
|----------|----------------|
| CATEGORY | Best practices |
| LOCATION | Global         |
| REVIEW   | Main Review    |

## DESCRIPTION

The `BracketVault`, `BracketOracle` and `BrktETH`, contracts are currently marked as unlicensed, as indicated by the SPDX license identifier at the top of the file:

```
SPDX-License-Identifier: UNLICENSED
```

Using unlicensed contracts can lead to legal uncertainties and conflicts regarding the usage, modification and distribution rights of the code.

## RECOMMENDATION

It is recommended to choose and apply an appropriate open-source license to the smart contract. Some options are:

1. MIT License: A permissive license that allows for reuse with minimal restrictions.
2. GNU General Public License (GPL): A copyleft license that ensures derivative works are also open-source.
3. Apache License 2.0: A permissive license that provides an express grant of patent rights from contributors to users.

## RESOLUTION

Status: Resolved.

LOW **L-11**

ACKNOWLEDGED

# balanceOf Includes Pending Amounts

CATEGORY Unexpected behavior  
LOCATION RebasingToken.sol: 17, 21  
REVIEW Main Review

## DESCRIPTION

The `balanceOf` function relies on the `sharesOf` to determine the balance of a user. The `sharesOf` function includes the `pendingDepositShares` which can include funds that the user has not yet been minted from the `lastDeposit` and may not be able to presently mint in the case where the `lastDeposit.epoch == epoch`.

As a result the `balanceOf` and `sharesOf` function reflects funds that are not available to the user and may lead to confusion for users and integrators.

## RECOMMENDATION

Be aware of this behavior, if it is expected then be sure to document this clearly for users and integrators.

## RESOLUTION

Status: Acknowledged.

LOW **L-12**

RESOLVED

# Vanity Nav Updated In Epoch 0

CATEGORY Validation  
LOCATION BracketVault.sol: 154  
REVIEW Main Review

## DESCRIPTION

The `updateVanityNav` function may be called to set the `vanityNav` to something other than `1e18` before the vault has been started with the `startVault` function.

This may lead to unexpected behavior and should not be a supported interaction.

## RECOMMENDATION

Consider validating that the epoch is greater than 0 in the `updateVanityNav` function.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/664a697224c2e9c6cba995402d784214e5d2792>

LOW **L-13**

RESOLVED

# Lacking Pause Mechanism

|          |                |
|----------|----------------|
| CATEGORY | Best practices |
| LOCATION | BrktETH.sol    |
| REVIEW   | Main Review    |

## — DESCRIPTION

The owner of `BrktETH` is able to blacklist collaterals, preventing any further deposits. However, this does not prevent collateral withdrawals.

If one of the external protocols is compromised, it will create a bank run, as users will try to withdraw the other collaterals not affected. In this case, a pausing mechanism will help to address the situation and implement the appropriate fixes.

Pausing the protocol can also allow owner to prevent user actions during an upgrade or fix.

## — RECOMMENDATION

Consider adding a pausing mechanism, inheriting from OZ `PausableUpgradeable` and implement the admin functions `pause` and `unpause`.

## — RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/003f154510a0eea3cb25c23ebe1befcd2cef0498>

LOW **L-14**

RESOLVED

# Unnecessary Address(0) Check

CATEGORY Optimization  
LOCATION BrktEth.sol: 53  
REVIEW Main Review

## DESCRIPTION

In the `initialize` function there is an address 0 check against all entries of the `tokens` array, however the `addCollateral` function already implements such a check.

## RECOMMENDATION

Remove the redundant address zero check in the `initialize` function for the `tokens` entries.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/c519f908a81f9d117454c39fa449beab7a1fd5f8>

LOW **L-15**

RESOLVED

## Excessive Wait For Manager Withdrawals

CATEGORY Logical Error  
LOCATION BrktETH.sol#L23  
REVIEW Main Review

### — DESCRIPTION

The `manager` will use `brktETH` from vault depositors for certain investment strategies. Therefore, the `brktETH` tokens will need to be burned to extract the underlying collateral.

The issue relies on the withdrawal mechanism, as users will need to wait 5 days to be able to claim the collateral. This creates a barrier for the manager and affects investments during this waiting period.

### — RECOMMENDATION

Consider adding an exception for managers, either receiving the collateral immediately or reduce the waiting period.

### — RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/dcf04df617a0e7745e433ec69458d331ef0a98ab>

LOW **L-16**

RESOLVED

## Collateral DoS Risk

|          |                  |
|----------|------------------|
| CATEGORY | DoS              |
| LOCATION | BrktEth.sol: 203 |
| REVIEW   | Main Review      |

### — DESCRIPTION

The `addCollateral` function does not validate a maximum number of collateral tokens that can be added to the array.

As a result the owner may on accident add too many collateral tokens over time which must all be looped over in the `getTotalValue` function. This may cause operations to require more than the block gas limit or at least be quite expensive to operate.

### — RECOMMENDATION

Consider adding a validation against a maximum number of supported collaterals. Additionally, consider implementing functionality to be able to remove collateral tokens from the `collaterals` list when they are blacklisted and have no deposits so that this does not become an issue over time.

### — RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/1bdc15e1d0eef2ab9bcda6db31601134475b1520>

LOW **L-17**

RESOLVED

# Lacking Slippage Check Allows Griefing

CATEGORY Griefing  
LOCATION brktEth.sol: 155  
REVIEW Main Review

## DESCRIPTION

In the `swapCollateral` function there is no way for the caller to specify the worst rate they are willing to accept for a swap.

There is no slippage at the bracket level, however the rate reported by some tokens may be manipulated by a malicious actor who wishes to grief the owner who is making the swap through bracket.

For example, the Renzo EZEth price may be manipulated by force sending Ether to the `depositQueue` and `withdrawQueue` address.

<https://github.com/Renzo-Protocol/contracts-public/blob/master/contracts/RateProvider/BalancerRateProvider.sol#L31>

<https://github.com/Renzo-Protocol/contracts-public/blob/9be7ca8619adc089dcd533a6796e1750d23061b5/contracts/RestakeManager.sol#L429>

## RECOMMENDATION

Consider adding a `maxAmountIn` parameter to the `swapCollateral` function to allow the caller to protect themselves from any potential griefing attacks that may apply to any arbitrary collateral token.

## RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/d016167010a3cceb35fe915236ac44c181dd>

HIGH **H-01**

ACKNOWLEDGED

# Missing collateralsIndex Update

CATEGORY Logical Error  
LOCATION BrktEth.sol: 144  
REVIEW Remediation Review

## DESCRIPTION

In the `removeCollateral` function the `collateralsIndex` entry for the token that is moved from the back of the list to replace the token being moved is not updated. Furthermore the index entry of the `collateralsIndex` for the token being removed is not cleared.

This will perturb the accounting of the system and validation of collaterals.

## RECOMMENDATION

Delete the entry for the token being removed from the `collateralsIndex` mapping.

Furthermore update the `collateralsIndex` entry for the token at the back of the list that is being moved to overwrite the removed token.

The `removeCollateral` function should look as follows:

```
function removeCollateral(address token) external onlyRole(DEFAULT_ADMIN_ROLE) {
    uint256 index = _getIndex(token);
    if (collaterals[index].whitelisted) revert CollateralNotBlacklisted();
    if (collaterals[index].totalDeposit != 0) revert CollateralNotEmpty();
    address moveToken = collaterals[collaterals.length - 1];
    collaterals[index] = moveToken;
    collateralsIndex[moveToken] = index + 1;
    delete collateralsIndex[token];
    collaterals.pop();
}
```

## RESOLUTION

Status: Acknowledged.

<https://github.com/bracket-fi/core-contracts/commit/dc0e78ec0e04eab9820048bb824afa417b29ede0>

MEDIUM **M-01**

ACKNOWLEDGED

# Invalid Index Checks

|          |                    |
|----------|--------------------|
| CATEGORY | Logical Error      |
| LOCATION | BrktEth.sol        |
| REVIEW   | Remediation Review |

## DESCRIPTION

Throughout the `BrktEth` contract several invalid index checks are made which errantly assume that a 0 index indicates that the token is not a supported collateral.

In the `whitelistCollateral`, `blacklistCollateral`, and `removeCollateral` functions the result of the `_getIndex` is compared against 0 to see if the collateral token has been configured. However this is incorrect as the returned index from the `_getIndex` function is already reduced by 1.

## RECOMMENDATION

Remove the errant index 0 check in the `whitelistCollateral`, `blacklistCollateral`, and `removeCollateral` functions.

## RESOLUTION

Status: Acknowledged.

<https://github.com/bracket-fi/core-contracts/commit/18b9d319db9ca99bb3123363bb917df078c8ff0c>

MEDIUM **M-02**

ACKNOWLEDGED

# lastNavUpdate Updated Before Conditional Check

CATEGORY Logical Error  
LOCATION BracketVault.sol  
REVIEW Remediation Review

## DESCRIPTION

The lastNavUpdate variable is updated to block.timestamp before the conditional check that uses this variable. Consequently, the condition `lastNavUpdate + NAV_NO_CONFIRMATION_PERIOD > block.timestamp` will always evaluate to true. As a result, only accounts with the NAV\_CONFIRMER\_ROLE will be able to call the updateNav function.

## RECOMMENDATION

Update lastNavUpdate only after the condition is validated.

## RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

# Missing Libraries Initialization

|          |                    |
|----------|--------------------|
| CATEGORY | Best Practices     |
| LOCATION | Global             |
| REVIEW   | Remediation Review |

## DESCRIPTION

Several upgradeable contracts inherit OpenZeppelin contracts/libraries that require calling their respective `__[LibraryName]_init` functions inside the contract's own `initialize` function.

Although some contracts do make certain initializer calls (e.g. `__Ownable_init(msg.sender)`, `__ERC20Permit_init("Bracket Liquid Staked ETH")`, `__ERC20_init(...)`, etc.), others do not consistently call all the initializers of their inherited contracts. For example:

- `BracketPricing` inherits `UUPSUpgradeable` but never calls `__UUPSUpgradeable_init()`.
- `BrktETH` inherits `UUPSUpgradeable`, `PausableUpgradeable`, etc. and only partially initializes some of them (e.g., `__AccessControlDefaultAdminRules_init` is called, but there is no corresponding call to `__UUPSUpgradeable_init()` or `__Pausable_init()`).

## RECOMMENDATION

Review each upgradeable contract's parent classes to identify all required initializers (e.g. `__UUPSUpgradeable_init`, `__Ownable_init`, `__AccessControl_init`, `__Pausable_init`, etc.) and ensure they are properly invoked in the contract's `initialize` function.

## RESOLUTION

Status: Acknowledged.

LOW **L-01.2**

ACKNOWLEDGED

# Typo

|          |                    |
|----------|--------------------|
| CATEGORY | Typo               |
| LOCATION | BrktEth.sol: 24    |
| REVIEW   | Remediation Review |

## DESCRIPTION

In the `BrktEth` contract the `INITIAL_ADMIN_CHNAGE_DELAY` variable contains a typo.

## RECOMMENDATION

Correct it to `INITIAL_ADMIN_CHANGE_DELAY`.

## RESOLUTION

Status: Acknowledged.

<https://github.com/bracket-fi/core-contracts/commit/6a885d0fa3a964c12516c42ac06ab2e06e9f8140>

LOW **L-02**

ACKNOWLEDGED

# Unimplemented `_getPythPrice` Function

|          |                        |
|----------|------------------------|
| CATEGORY | Warning                |
| LOCATION | BracketPricing.sol: 63 |
| REVIEW   | Remediation Review     |

## — DESCRIPTION

In the `BracketPricing` contract, the function `_getPythPrice` is declared but not implemented, making any `Oracle.Pyth` configuration inoperable.

## — RECOMMENDATION

Implement `_getPythPrice` to properly fetch and decode the Pyth price.

## — RESOLUTION

Status: Acknowledged.

LOW **L-02.2**

ACKNOWLEDGED

## Custom Revert Will Not Be Reached

|          |                    |
|----------|--------------------|
| CATEGORY | Warning            |
| LOCATION | BrktETH.sol        |
| REVIEW   | Remediation Review |

### — DESCRIPTION

The validation `if (oracle.getRate(token) == 0) revert CollateralNotSupported();` was added to `addCollateral` to ensure the oracle supports the collateral being added.

However, if the token is not supported then `oracle.getRate(token)` would revert with `InvalidToken()` and the `CollateralNotSupported` revert would not be triggered.

### — RECOMMENDATION

Be aware of this behavior.

### — RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

# **\_getChainlinkPrice Function Is Only Compatible With 8 decimals Price Feeds**

CATEGORY Warning  
LOCATION BracketPricing.sol: 60  
REVIEW Remediation Review

## — DESCRIPTION

The `_getChainlinkPrice` function in `BracketPricing` contract multiplies the price by `1e10` to convert it to 18 decimals (`uint256(price) * 1e10`). This works only if the Chainlink aggregator returns an 8-decimal price. If an aggregator uses any other number of decimals, the result becomes incorrect.

## — RECOMMENDATION

Retrieve the feed's actual number of decimals via `priceFeed.decimals()` and scale the value accordingly. A more robust example:

```
function _getChainlinkPrice(address oracle) internal view returns (uint256) {
    AggregatorV2V3Interface priceFeed = AggregatorV2V3Interface(oracle);
    uint8 decimals = priceFeed.decimals();
    (
        ,
        int256 price,
        ,
        ,
    ) = priceFeed.latestRoundData();
    if (price <= 0) revert ZeroPrice();
    // Convert the price to 18 decimals
    uint256 scaledPrice = uint256(price);
    if (decimals < 18) {
        scaledPrice *= 10 ** (18 - decimals);
    } else if (decimals > 18) {
        scaledPrice /= 10 ** (decimals - 18);
    }
    return scaledPrice;
}
```

---

**L-03**    `_getChainlinkPrice` Function Is Only Compatible With 8 decimals Price Feeds    **CONTINUED**

---

— **RESOLUTION**

Status: Acknowledged.

## § 05

# Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

# About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

**\$45B+**DIGITAL ASSETS UNDER  
REVIEW**200+**

ENGAGEMENTS COMPLETED

**500+**CRITICAL & HIGH FINDINGS  
REPORTED

Reach us at [audits@guardianaudits.com](mailto:audits@guardianaudits.com).